An Investigation into Action Space Constraints in Reinforcement Learning

by

Francois Qian
URN: 6702759/7

A dissertation submitted in partial fulfilment of the
requirements for the award of

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

May 2026

Department of Computer Science
University of Surrey
Guildford GU2 7XH

Supervised by: Nishanth Sastry

I declare that this dissertation is my own work and that the work of others is acknowledged and indicated by explicit references.

Francois Qian
May 2026

© Copyright Francois Qian, May 2026

Abstract

Acknowledgements

I would like to thank my supervisor, Nishanth Sastry, for overseeing the project.

Contents

1 Introduction **14**
1.1 Context and Motivation 14
1.2 Aims and Objectives 14
1.3 Risk Management 15

2 Literature Review **16**
2.1 Chess Fundamentals 16
2.1.1 Game Rules and State Space 16
2.1.2 Board Representation and Move Generation 16
2.2 Machine Learning in Chess 16
2.2.1 Reinforcement Learning and MCTS 16
2.2.2 Transformers and Action Spaces 16
2.2.3 Grokking and Masking 16

3 Methodology and Hypotheses **17**
3.1 H1: Ruleset Convergence (Pseudo-Legal vs. Legal) - **Control:** Training on a strict legal ruleset. 17
3.2 H2: Logit Masking vs. Punishment Propagation 17
3.3 H3: Two-Pass Autoregressive Encoder 17

4 System Architecture and Implementation **18**
4.1 Client Architecture (Rust) 18
4.1.1 Bitboard Implementation 18
4.1.2 Move Generation 18
4.2 Model Architecture 18
4.2.1 Inputs and Outputs 18
4.2.2 Two-Pass Execution Flow 19

5 Training and Self-Play **20**
5.1 Monte Carlo Tree Search 20
5.2 Self-Play Pipeline 20

6 Results and Evaluation **21**
6.1 Experimental Setup 21
6.2 H1: Ruleset Convergence Analysis 21
6.3 H2: Masking and Grokking Analysis 21

7 Conclusion and Future Work **22**
7.1 Summary of Findings 22
7.2 Limitations and Future Work 22
7.3 Using Typst 23
7.3.1 Adding figures 23
7.3.2 Adding tables 23
7.3.3 Adding equations 23
Bibliography **25**

List of Figures

Figure 1 An example figure, with two parts 24

List of Tables

Table 1 **Comparative performance metrics across the four experimental variants.** **21**

Table 2 **An example table, showing speed in operations per cycle per multiprocessor** **23**

Glossary

$Q(s, a)$	The total reward accumulated from state
$C(s)$	CPUCT
S_i	Upper confidence bound
$\overline{x_i}$	The Mean Value of the current node
c	The Exploitation parameter
N	The current node's Visit Count
n_i	The Visit Count of the child

Abbreviations

MCTS	Monte Carlo Tree Search
UCB1	Upper Confidence Bound 1
PUCT	Predictor + Upper Confidence Bound applied to Trees
CPUCT	Exploration Predictor Constant

Chapter 1

Introduction

1.1 Context and Motivation

Modern machine learning often relies on scaling data and compute over opaque architectures. This project deconstructs the learning dynamics of transformer models by introducing a novel architecture and evaluating emergent behaviors within unrestricted action spaces. Chess serves as the ideal environment: it presents highly ambiguous intermediate states bound by objective ground truths and a mathematically perfect, yet computationally intractable, solution space.

This research investigates four primary areas:

1. The convergence of self-play in a pseudo-legal environment toward strict legal play.
2. The impact of logit masking on learning dynamics, specifically regarding the "grokking" of physical board mechanics.
3. The efficacy of a two-pass autoregressive encoder in reducing the action space within a Monte Carlo Tree Search

(MCTS) framework.

1.2 Aims and Objectives

The primary aim is to train multiple transformer-based agents via self-play reinforcement learning across varying rule sets (legal vs. pseudo-legal) and action spaces (masked vs. unmasked logits), evaluating their performance and learning trajectories.

Specific objectives include:

- Developing a high-performance, bespoke chess client in Rust supporting both pseudo-legal and legal state generation.
- Implementing a custom move generator utilizing bitboards, bitwise intrinsics, and Zobrist hashing.
- Architecting a two-pass autoregressive transformer encoder tightly integrated with the engine.
- Constructing a bipartite MCTS algorithm for self-play data generation.
- Training four model variants and capturing leading metrics (policy entropy, illegal move probability, estimated ELO).

1.3 Risk Management

The primary risk is scope creep, specifically regarding the bespoke Rust client implementation. This is mitigated through strict modularity, rigorous unit testing, and limiting the scope of the engine to the requirements of the neural network (e.g., deferring Endgame Tablebase integration to future work).

Chapter 2

Literature Review

2.1 Chess Fundamentals

2.1.1 Game Rules and State Space

Chess is a zero-sum, perfect-information game with a state-space complexity of 10^{40} and a game-tree complexity of 10^{120} (Shannon Number). Traditionally, engines operate under strict legal constraints. This investigation introduces "pseudo-legal" environments —ignoring king safety constraints—to observe if transformers implicitly learn higher-order rules (pins, checks) via punishment propagation.

2.1.2 Board Representation and Move Generation

Representation has evolved from piece-lists to bitboards (64-bit integers). Bitboards enable high-performance move generation via bitwise intrinsics. In deep learning, these are expanded into $8 \times 8 \times N$ tensors to suit the inductive biases of the network.

2.2 Machine Learning in Chess

2.2.1 Reinforcement Learning and MCTS

Review of AlphaZero's integration of MCTS with policy/value networks [1].

2.2.2 Transformers and Action Spaces

Review of recent applications of attention mechanisms in chess [2].

2.2.3 Grokking and Masking

Analysis of delayed generalization ("grokking") [3] and the standard practice of logit masking in AlphaZero-style architectures.

Chapter 3

Methodology and Hypotheses

3.1 H1: Ruleset Convergence (Pseudo-Legal vs. Legal) - Control: Training on a strict legal ruleset.

- **Test:** Training on a pseudo-legal ruleset (explicit king capture terminates the game).
- **Hypothesis:** Pseudo-legal training naturally converges to legal play. Checkmates and pins will manifest as emergent

behaviors to secure or prevent king capture.

3.2 H2: Logit Masking vs. Punishment Propagation

- **Control:** Masking illegal moves prior to softmax (network is blind to rules).
- **Test:** Unmasked logits. Illegal moves are punished via the loss function.
- **Hypothesis:** Unmasked training will initially learn slower but forces the network to internally map ("grok") the

physical mechanics of the board, potentially yielding more robust long-term representations.

3.3 H3: Two-Pass Autoregressive Encoder

- **Hypothesis:** Decomposing the action space into a two-pass selection (origin square, then destination square)

provides a more human-like decision-making approach. While this doubles MCTS tree depth, it significantly reduces the model's immediate action space complexity.

Chapter 4

System Architecture and Implementation

4.1 Client Architecture (Rust)

A bespoke client was developed in Rust to meet the high-performance demands of MCTS rollouts and ensure memory safety during concurrent self-play.

4.1.1 Bitboard Implementation

The board is represented using 12 primary bitboards (unsigned 64-bit integers), packing the state into 96 bytes. Additional convenience bitboards (occupancy masks) are maintained to accelerate move generation.

4.1.2 Move Generation

Ray casting is optimized using constant directional masks and pre-computed "between" masks, allowing for $O(1)$ validation of sliding piece moves.

```rust
// Core API Structure
pub struct ChessGame {
    pub castling_rights: CastlingRights,
    pub chessboard: ChessBoard,
    pub en_passant: Option<ChessSquare>,
    pub fullmove_counter: u32,
    pub game_history: alloc::vec::Vec<GameStateEntry>,
    pub halfmove_clock: u32,
    pub outcome: Outcome,
    pub rule_set: RuleSet,
    pub side_to_move: chess_piece::Color,
    pub zobrist_hash: u64,
}
```

4.2 Model Architecture

4.2.1 Inputs and Outputs

- **Inputs:** 12 piece bitboards, 1 en passant bitboard, 1 selected square bitboard, castling rights (4x1 f32), 50-move

rule scalar (1 f32).

- **Outputs:** Policy head (64-square distribution) and Value head (Win/Draw/Loss buckets). Both utilize softmax

activation and KL divergence loss.

4.2.2 Two-Pass Execution Flow

1. **Pass 1:** Select origin square.
2. **Pass 2:** Select destination square (Pass 1 output fed back into input). Promotions are handled automatically during

MCTS rollouts.

Chapter 5

Training and Self-Play

5.1 Monte Carlo Tree Search

The MCTS utilizes the PUCT (Predictor + Upper Confidence Bound applied to Trees) algorithm to balance exploration and exploitation.

$$\alpha(\iota, x_2) = \sum_{x_1, d_{\iota-1}} \alpha(\iota - 1, x_1) \Pr\left\{ \boldsymbol{r}_{n(\iota-1)+x_1, n\iota+x_2}, \boldsymbol{t}_{\iota-1} \right\}$$

5.2 Self-Play Pipeline

Data is generated asynchronously using Rayon. The network is updated via backpropagation using autograd, minimizing the combined policy and value loss.

Chapter 6

Results and Evaluation

6.1 Experimental Setup

To ensure fairness, all variants were initialized with identical seeds, trained for the same number of epochs, and evaluated at identical MCTS search depths.

Model Variant	Peak ELO	Illegal Move %
Legal Masked (Control)	TBD	0.0%
Legal Unmasked	TBD	TBD%
Pseudo-Legal Masked	TBD	0.0%
Pseudo-Legal Unmasked	TBD	TBD%

Table 1: Comparative performance metrics across the four experimental variants.

6.2 H1: Ruleset Convergence Analysis

[Insert analysis of pseudo-legal vs legal performance]

6.3 H2: Masking and Grokking Analysis

[Insert analysis of policy entropy and illegal move probability over time]

Chapter 7

Conclusion and Future Work

7.1 Summary of Findings

This project successfully implemented a bespoke Rust chess engine and a novel two-pass transformer architecture. Empirical validation of the four experimental branches provided insights into [insert core finding regarding masking/rulesets].

7.2 Limitations and Future Work

- **Compute Constraints:** Hardware limitations restricted the total number of training epochs, preventing observation of

deep late-stage grokking.

- **Move Generation:** Future iterations should replace standard bitboard raycasting with Magic Bitboards for optimal

performance.

- **Endgame Tablebases (EGTB):** Injecting perfect knowledge at varying stages of the training pipeline remains a highly

viable area for future research to bootstrap the value head and sharpen endgame play.

7.3 Using Typst

Operation	Speed
Add, Mul, Mul-Add	8
Reciprocal	2
Divide	0.88
Divide Intrinsic	1.6
Recip. Square Root	2
Square Root	1
Logarithm	2
Exponent	1
Sin, Cos Intrinsics	2
Sin, Cos, Tan	Slow

Table 2: An example table, showing speed in operations per cycle per multiprocessor

7.3.1 Adding figures

Figures are added using the `#figure` function. Typst automatically handles the numbering and placement of your images, as shown in the example that produces Figure 1.1. Common formats like PNG, JPEG, and SVG are supported natively.

7.3.2 Adding tables

Tables are defined directly in the source code using the `#table` function. This provides a highly flexible way to grid your data. An example of a styled table is given in Table 1.1.

7.3.3 Adding equations

A primary advantage of Typst is its intuitive mathematical notation. Equations can be written within `$ ... $` delimiters. It handles numbered equations easily, as in the recursive formula:

$$\alpha(\iota, x_2) = \sum_{x_1, d_{\iota-1}} \alpha(\iota - 1, x_1) \Pr\left\{ \boldsymbol{r}_{n(\iota-1)+x_1, n\iota+x_2}, \boldsymbol{t}_{\iota-1} \right\}$$

Inline math is also supported, for example to specify $1 \leq \iota < N$. Typst's syntax remains readable even for complex multiline expressions: testing testings

Diagrammatic representation

(a) Diagrammatic representation

Photo of Cluster

(b) Photo of the Tempest cluster

Figure 1: An example figure, with two parts

$$\Pr\Big\{\boldsymbol{r}_{0,n\iota+x_2}, \sigma_{n\iota} = x_2\Big\} = \sum_{x_1, d_{\iota-1}} \Bigg[\Pr\Big\{\boldsymbol{r}_{0,n(\iota-1)+x_1}, \sigma_{n(\iota-1)} = x_1\Big\} \times \Pr\Big\{\boldsymbol{r}_{n(\iota-1)+x_1,n\iota+x_2}, \boldsymbol{t}_{\iota-1}\Big\}\Bigg]$$

Bibliography

[1] D. Silver *et al.*, "Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm." [Online]. Available: https://arxiv.org/abs/1712.01815

[2] D. Monroe and P. A. Chalmers, "Mastering Chess with a Transformer Model." [Online]. Available: https://arxiv.org/abs/2409.12272

[3] A. Power, Y. Burda, H. Edwards, I. Babuschkin, and V. Misra, "Grokking: Generalization beyond overfitting on small algorithmic datasets," *arXiv preprint arXiv:2201.02177*, 2022.